

El Viaje del Modelado

Una Guía Práctica de Machine Learning con Python

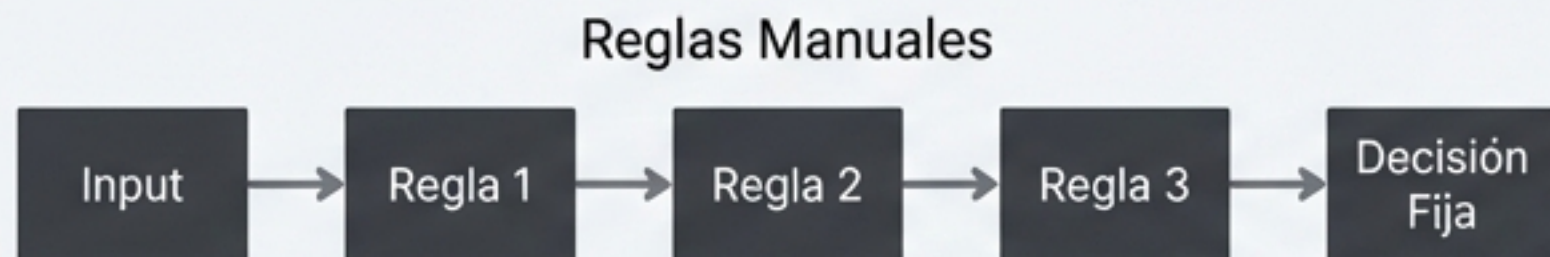


Basado en "Introduction to Machine Learning with Python" por Andreas C. Müller y Sarah Guido.

De Reglas Rígidas al Aprendizaje Automático

El objetivo del Machine Learning es **extraer conocimiento de los datos** para **automatizar decisiones**.

El Enfoque Tradicional

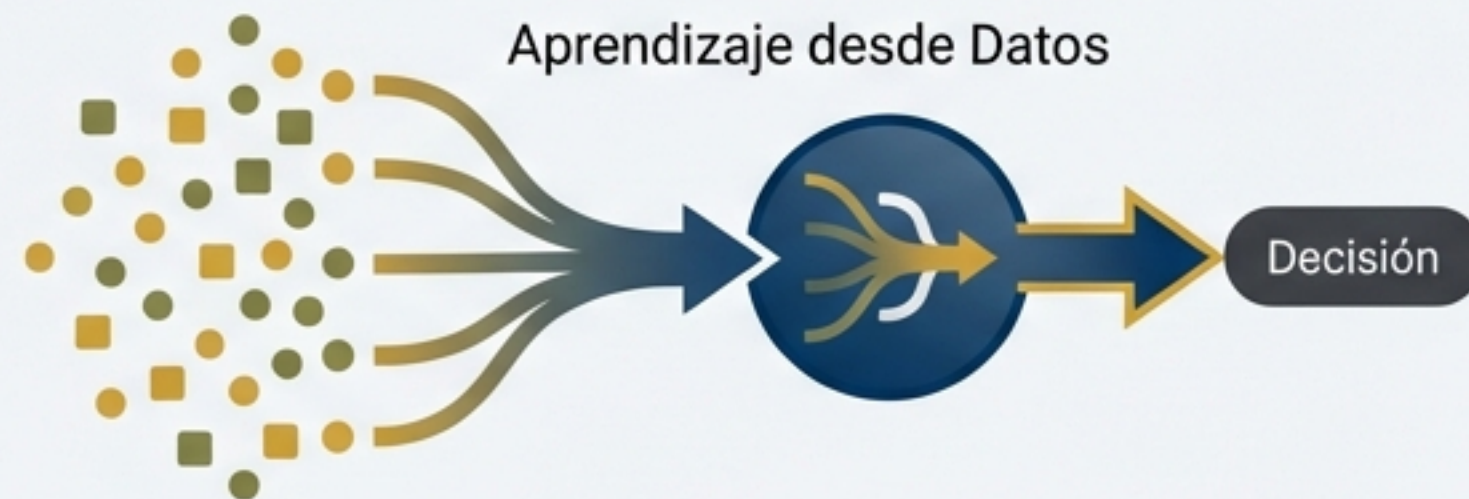


Sistemas basados en reglas `if/else` codificadas a mano.

Desventajas:

- La lógica es específica para un único dominio y tarea.
- Requiere un profundo conocimiento experto para diseñar las reglas.
- Fracasa en problemas complejos (ej. reconocimiento facial), donde las reglas son imposibles de definir para un humano.

El Enfoque de Machine Learning



El algoritmo aprende las reglas directamente de los ejemplos.

Ventajas:

- Generaliza a partir de ejemplos conocidos para tomar decisiones sobre datos nuevos.
- Puede resolver problemas donde las reglas son demasiado complejas de formular manualmente.
- Ejemplo: En lugar de definir qué es una "cara", le mostramos a un algoritmo miles de imágenes de caras y aprende las características por sí mismo.

El Ecosistema de Python: Nuestras Herramientas Esenciales

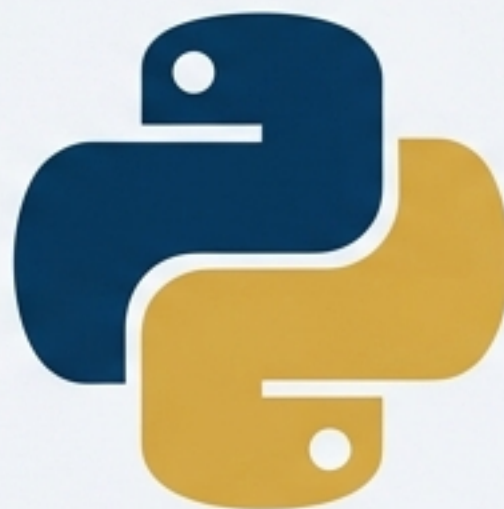
Python combina el poder de un lenguaje de propósito general con la facilidad de uso de lenguajes de scripting. El análisis de datos es un proceso iterativo, y Python permite una interacción y experimentación rápidas.



scikit-learn: La biblioteca central para Machine Learning en Python. Contiene algoritmos de última generación y una interfaz consistente.



pandas: Para la manipulación y análisis de datos. Construido sobre la estructura `DataFrame`, una tabla flexible y potente.

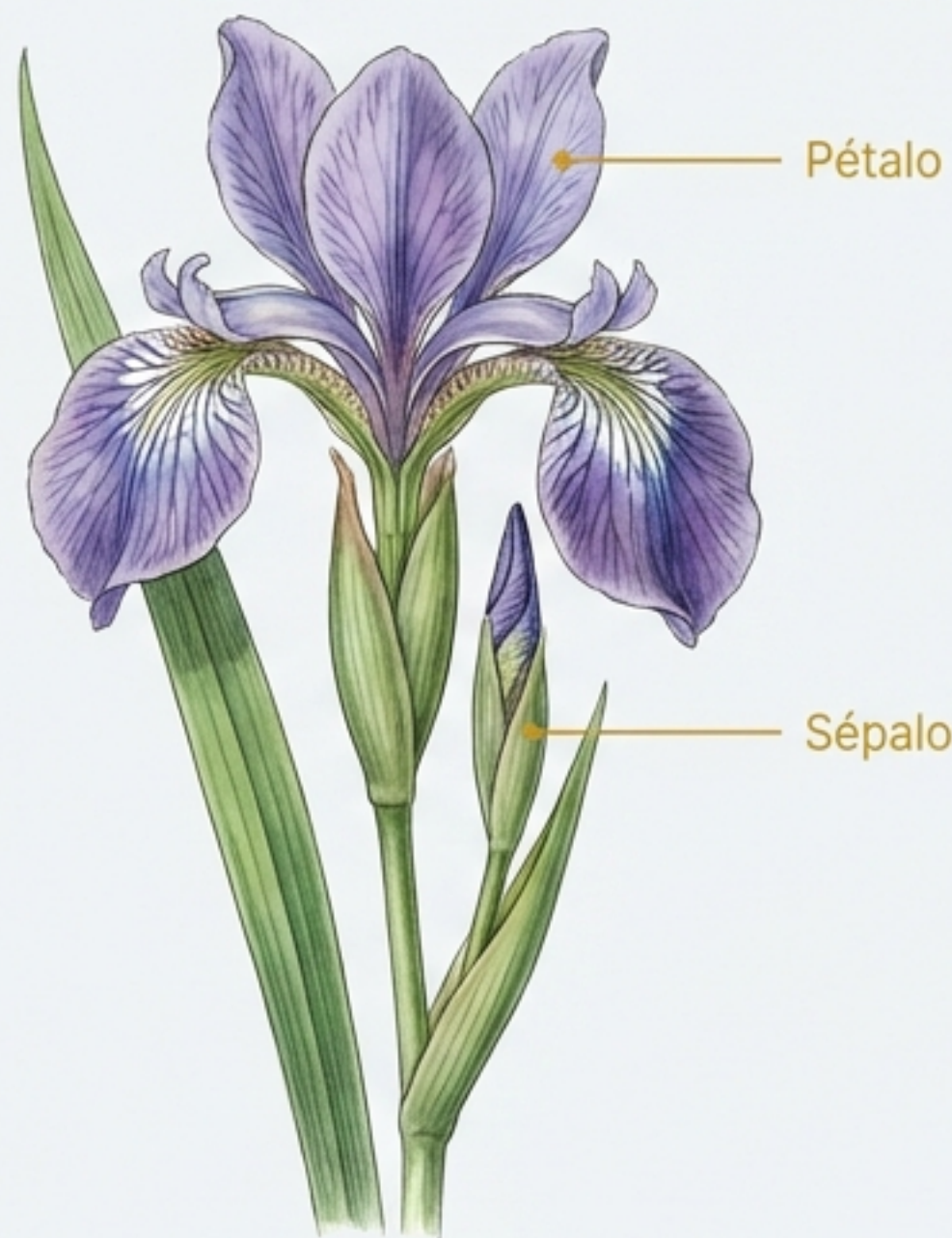


NumPy: El pilar fundamental. Proporciona la estructura de datos ``ndarray`` (array multidimensional) que scikit-learn utiliza como entrada.



matplotlib: La principal biblioteca para visualización científica. Esencial para inspeccionar datos y analizar resultados de modelos.

Nuestra Primera Aplicación: Clasificación de Especies de Iris



El Problema

Un botánico ha recolectado mediciones de flores de iris y quiere construir un modelo para predecir la especie de un nuevo ejemplar.

Tipo de Problema

Aprendizaje Supervisado (tenemos datos con las respuestas correctas). Específicamente, es un problema de clasificación multiclase.

Terminología Fundamental

- **Clases:** Las posibles salidas o categorías que queremos predecir (en este caso, las 3 especies: 'setosa', 'versicolor', 'virginica').
- **Etiqueta (Label):** La especie correcta para un punto de dato individual (una flor específica).
- **Muestra (Sample):** Un punto de dato individual (una fila en nuestra tabla de datos, representando una flor).
- **Característica (Feature):** Una propiedad que describe una muestra (una columna, ej. 'longitud del sépalo').

Lo Primero es lo Primero: Observar los Datos

Principio Clave

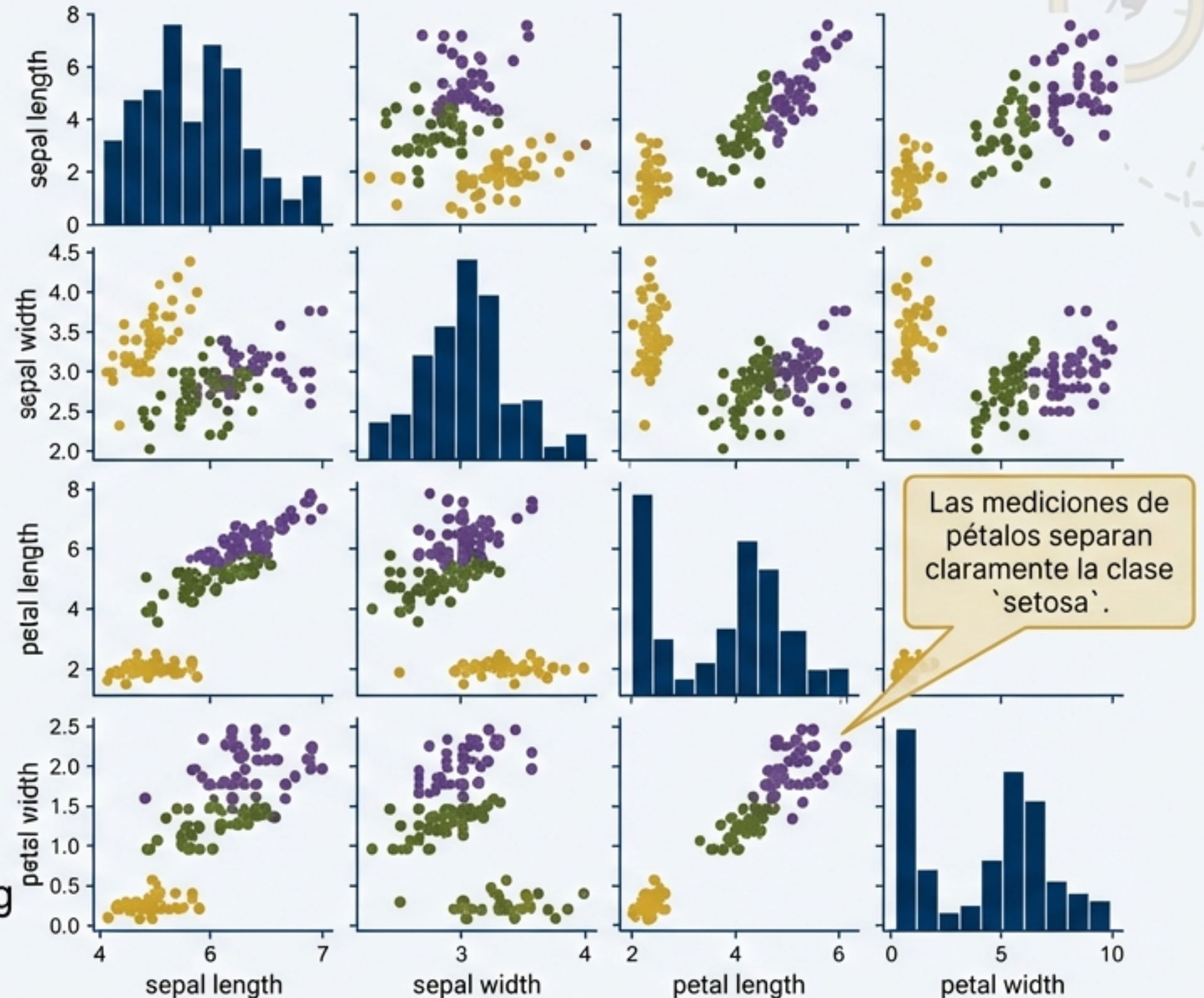
Antes de construir un modelo, la inspección visual es la mejor manera de entender los datos, encontrar anomalías y evaluar si la tarea es factible.

La Técnica

Un **gráfico de pares (pair plot)** nos permite visualizar todas las combinaciones posibles de características, dos a la vez. Los puntos se colorean según su clase (especie).

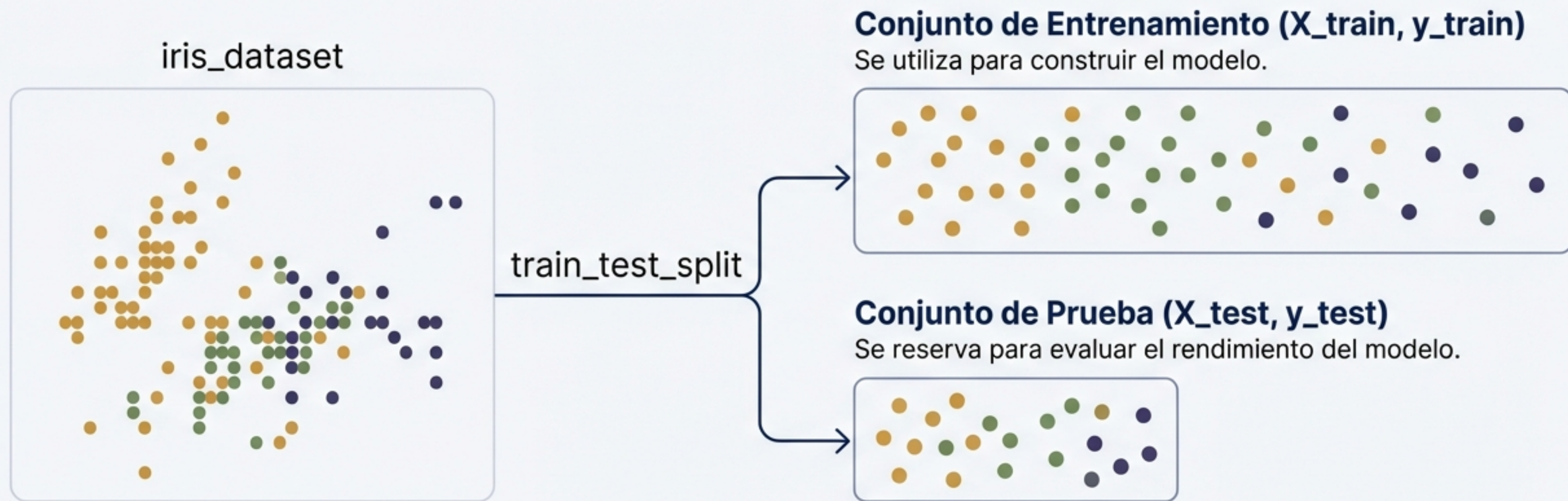
Análisis del Gráfico

- Las tres clases parecen estar relativamente bien separadas utilizando las mediciones de sépalos y pétalos.
- Esta separación visual nos da una fuerte indicación de que un modelo de machine learning probablemente podrá aprender a distinguirlas.



Midiendo el Éxito: Datos de Entrenamiento y Prueba

No podemos usar los mismos datos para construir y evaluar un modelo. El modelo podría simplemente 'memorizar' el conjunto de entrenamiento y no **generalizar** bien a datos nuevos y nunca antes vistos.



La función `'train_test_split'` de scikit-learn baraja el dataset y lo divide, asegurando que ambas partes contengan una mezcla representativa de todas las clases.

Construyendo y Evaluando el Primer Modelo: k-Vecinos más Cercanos (k-NN)

El Algoritmo y Proceso

El Algoritmo (k-NN)

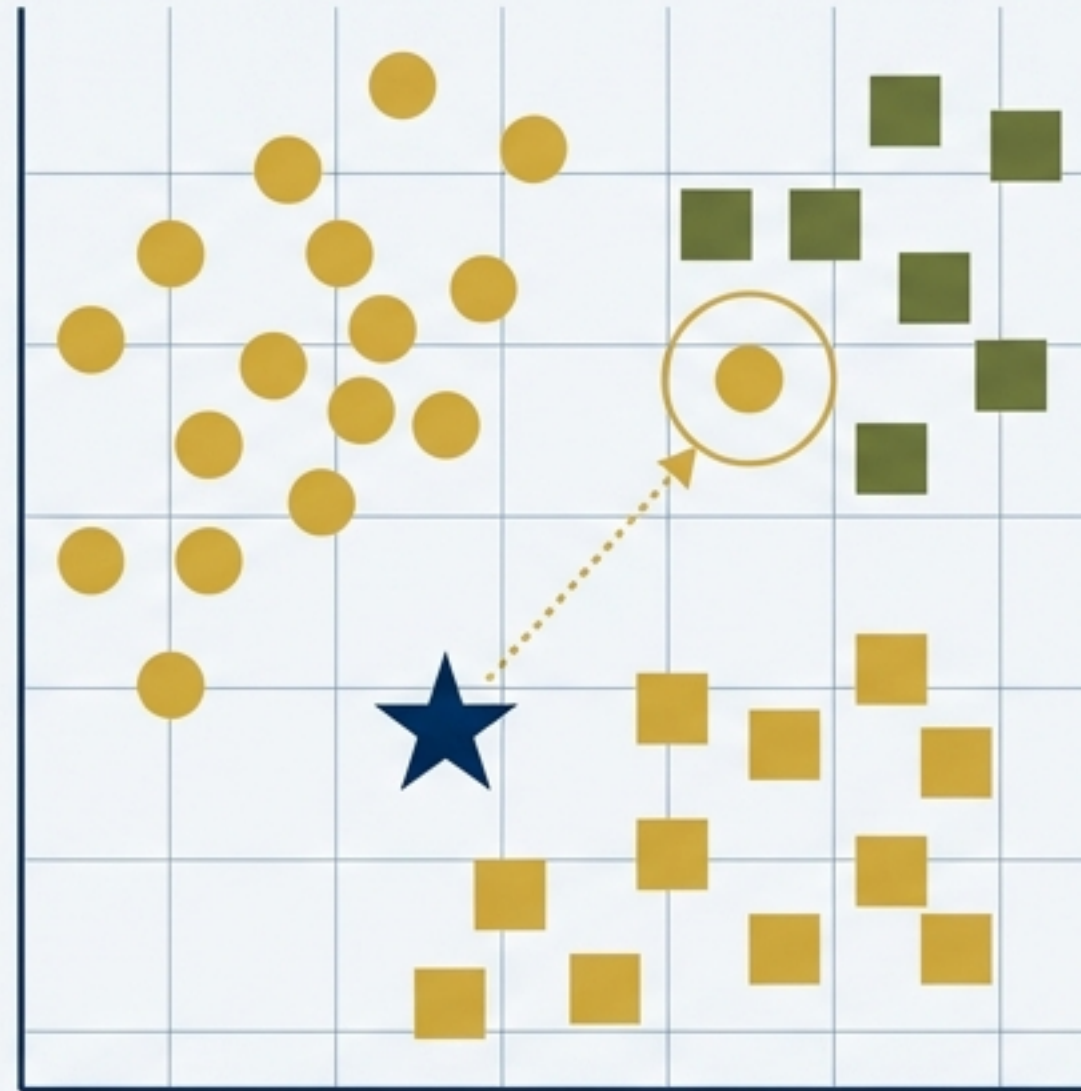
1. **Construcción (fit):** Simplemente almacena el conjunto de entrenamiento.
2. **Predicción (predict):** Para un nuevo punto, encuentra el punto más cercano ('k=1') en los datos de entrenamiento y le asigna su etiqueta.

El Proceso en scikit-learn

```
# 1. Instanciar
knn = KNeighborsClassifier(n_neighbors=1)

# 2. Entrenar
knn.fit(X_train, y_train)

# 3. Evaluar
knn.score(X_test, y_test)
```



El Resultado

Precisión en el conjunto de prueba:

97%

El modelo predijo correctamente el 97% de las iris en el conjunto de prueba. Este alto nivel de precisión nos da confianza para usarlo en nuevos datos.

El Dilema Central: Generalización, Sobreajuste y Subajuste

Queremos un modelo que **generalice** bien, es decir, que haga predicciones precisas sobre datos nuevos.



- **Subajuste:** El modelo es demasiado simple. No logra capturar la variabilidad de los datos.

- **Sobreajuste:** El modelo es demasiado complejo. Se ajusta demasiado a las particularidades de los datos de entrenamiento y no generaliza bien.

Un Arsenal de Algoritmos Supervisados

Ahora que entendemos el dilema central, exploraremos un conjunto de los algoritmos más utilizados en la práctica.

No veremos los algoritmos como una lista, sino como un conjunto de herramientas especializadas, cada una con sus propias fortalezas, debilidades y casos de uso ideales.



1. Modelos Lineales

La base de la simplicidad y la velocidad.



3. Ensembles (Bosques y Boosting)

El poder de la colaboración para superar debilidades.



2. Árboles de Decisión

Intuitivos y potentes, sin supuestos de linealidad.



4. Máquinas de Vectores de Soporte (SVM)

Maestría en el manejo de la complejidad no lineal.

La Base: Modelos Lineales y el Poder de la Regularización

El Mecanismo: Los modelos lineales hacen predicciones usando una función lineal de las características. En clasificación, el límite de decisión es una línea, un plano o un hiperplano.

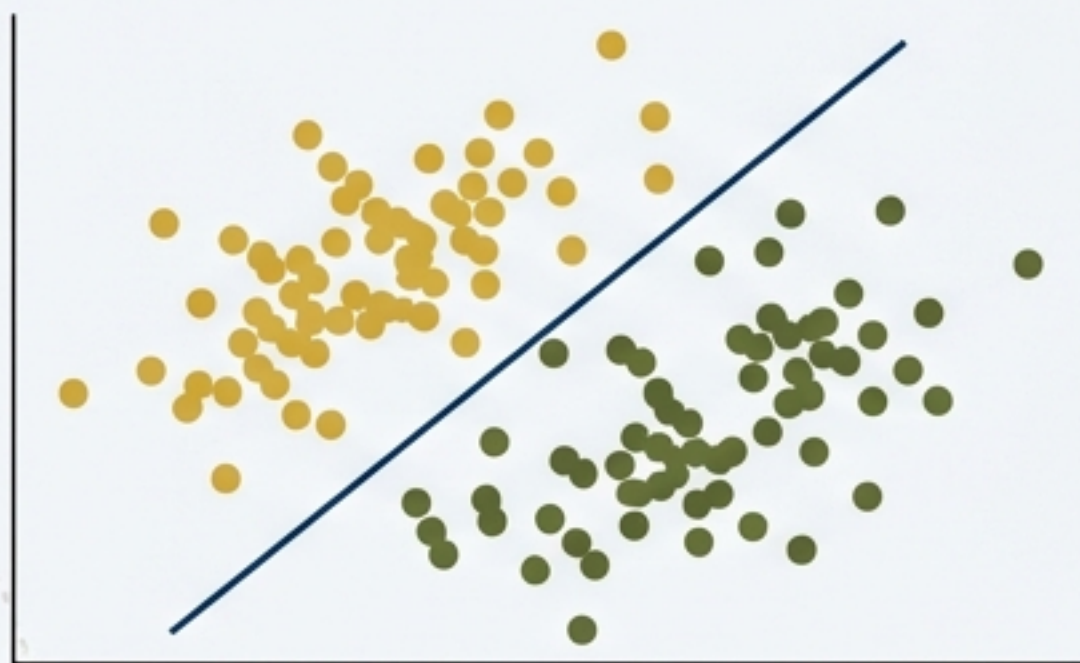
El Problema: En datasets con muchas características, pueden sobreajustar fácilmente.

La Solución: Regularización: Restringimos el modelo para evitar el sobreajuste, forzando a los coeficientes w a ser pequeños.

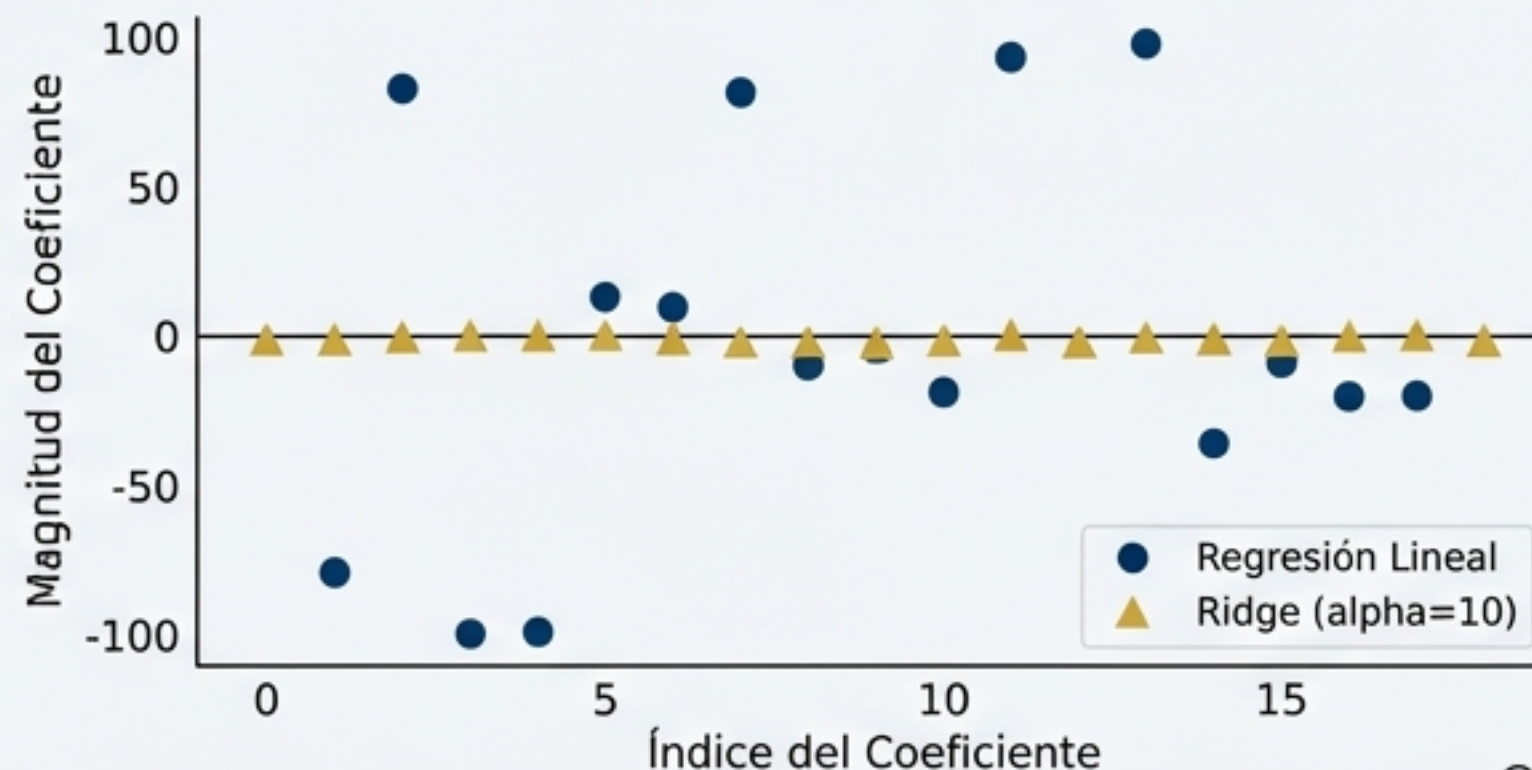
- **Ridge (L2):** Mantiene todos los coeficientes pequeños. Es la opción por defecto.
- **Lasso (L1):** Fuerza a algunos coeficientes a ser exactamente cero (selección de características).

Parámetro Clave: α o C controla la fuerza de la regularización.

Límite de Decisión Lineal



Regularización Ridge Reduce la Magnitud de los Coeficientes



Más Allá de la Linealidad: Árboles de Decisión

El Mecanismo: Los árboles aprenden una jerarquía de preguntas de tipo `if/`else` sobre las características características para llegar a una decisión.

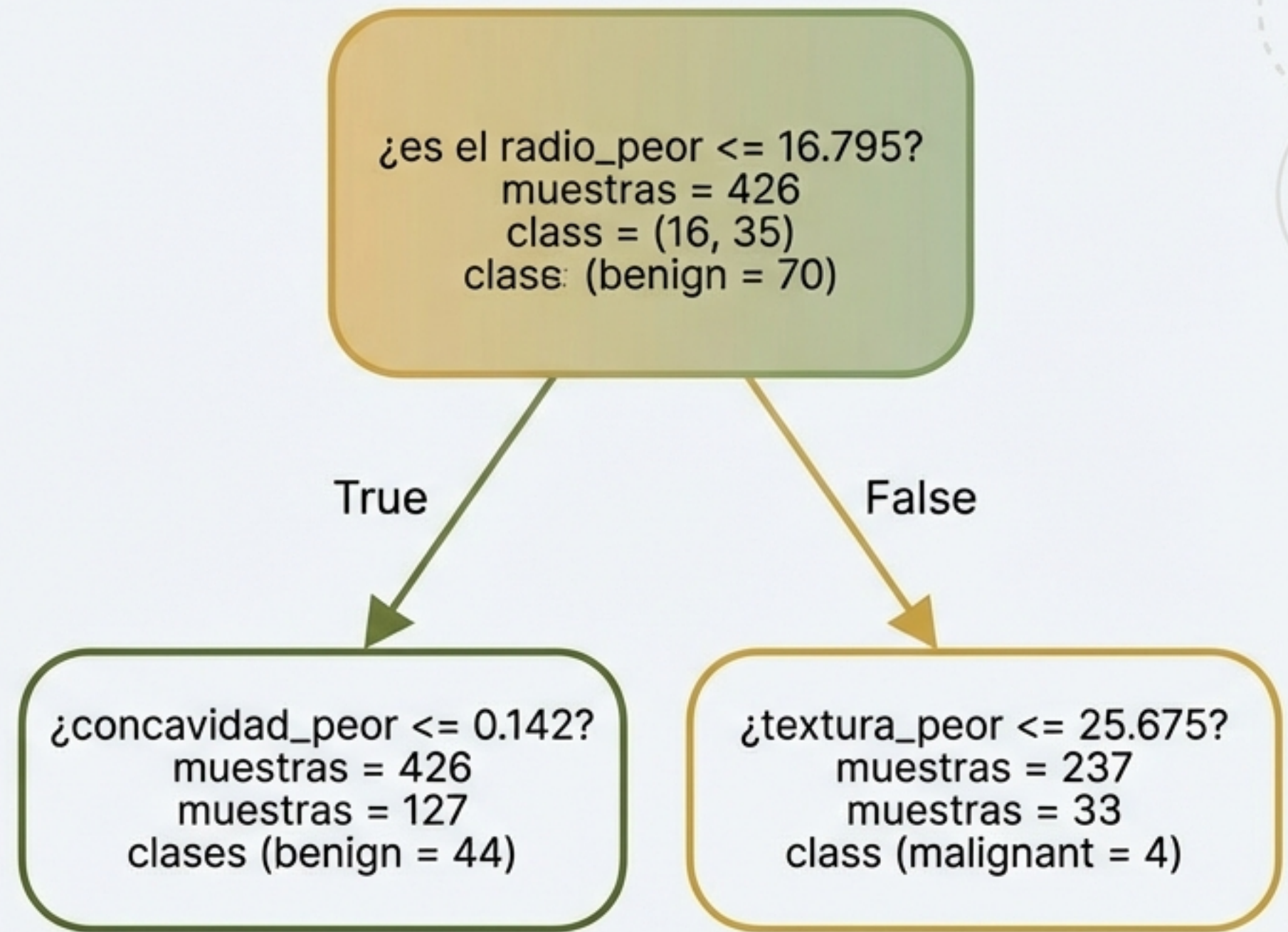
Cada nodo representa una "prueba" sobre una característica.

Fortalezas:

- El modelo resultante es fácil de visualizar y explicar.
- No requiere escalado de datos.

La Principal Debilidad:

- Los árboles individuales, si crecen sin restricciones, tienden a sobreajustar drásticamente.



Del Árbol al Bosque: El Poder de los Ensembles



El Problema: Sobreajuste

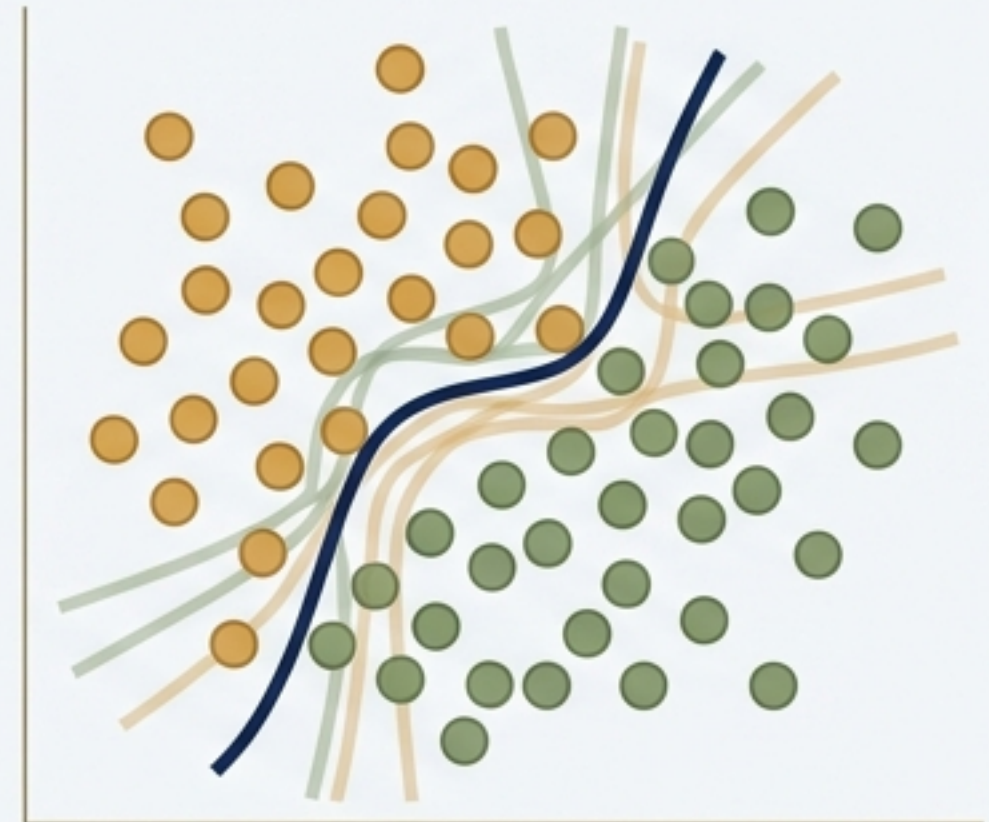


El Resultado: Robustez



La Solución: Random Forests

****Idea Clave**:** Si construimos muchos árboles que funcionan bien pero sobreajustan de maneras diferentes, podemos reducir el sobreajuste promediando sus resultados. Es la "sabiduría de la multitud".



****¿Cómo se logra la diversidad?****

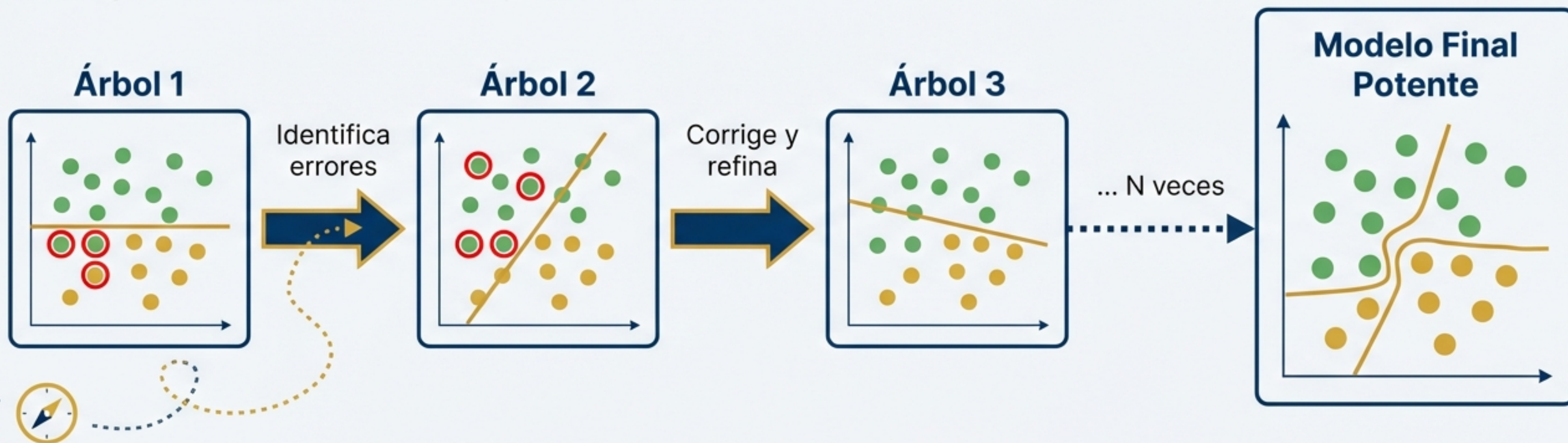
1. ****Bootstrap Sampling****: Cada árbol se entrena con una muestra ligeramente diferente de los datos.
2. ****Selección de Características****: En cada nodo, se elige un subconjunto aleatorio de características.

Aprendizaje Iterativo: Gradient Boosted Trees



Un Enfoque Diferente de Ensemble: A diferencia de los Random Forests que construyen árboles de forma independiente, Gradient Boosting los construye de manera **secuencial**.

La Idea Clave: Se construye una serie de árboles 'débiles', donde cada nuevo árbol intenta corregir los errores cometidos por el árbol anterior.

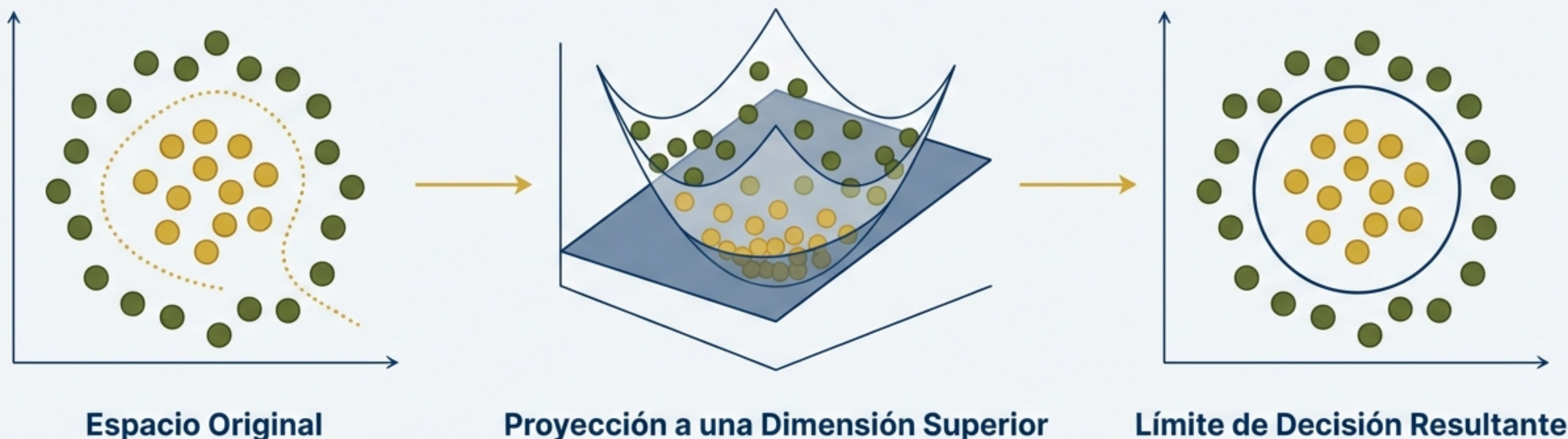


- A menudo es uno de los modelos con mayor rendimiento.
- Más sensible a los parámetros que los Random Forests (especialmente `n_estimators` y `learning_rate`).

Dominando la No Linealidad: Máquinas de Vectores de Soporte (SVM) con Kernel

El Desafío: ¿Qué pasa si los datos no se pueden separar con una línea recta?

La Solución: El 'Truco del Kernel': Un SVM con kernel mapea los datos a un espacio de mayor dimensión donde Sí se pueden separar linealmente.



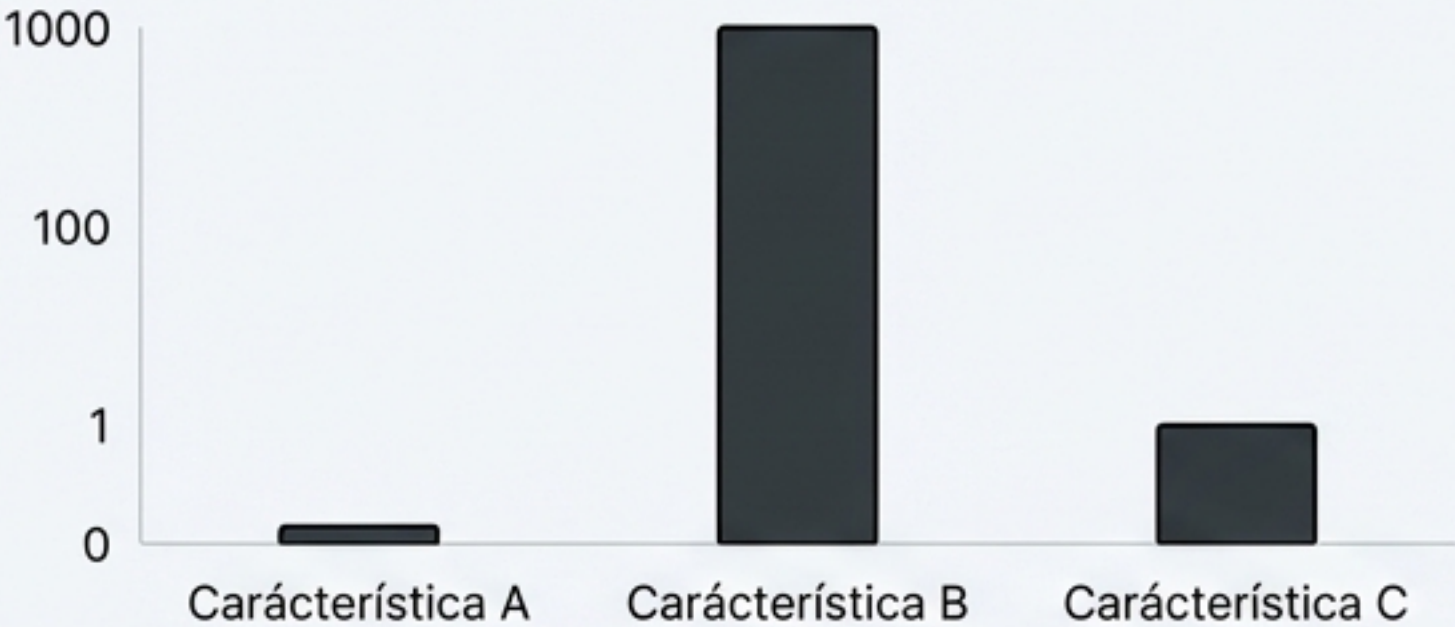
Resultado: Permite crear límites de decisión increíblemente complejos y no lineales.

Parámetros Clave: `'gamma'` (controla la complejidad del límite) y `'C'` (parámetro de regularización).

El Arte de la Preparación: El Impacto Crítico del Escalado de Datos

Algunos de los modelos más potentes, como los SVMs y las Redes Neuronales, son muy sensibles a la **escala de las características**. Si los rangos de las características son muy diferentes, el modelo puede verse afectado negativamente.

ANTES de Escalar



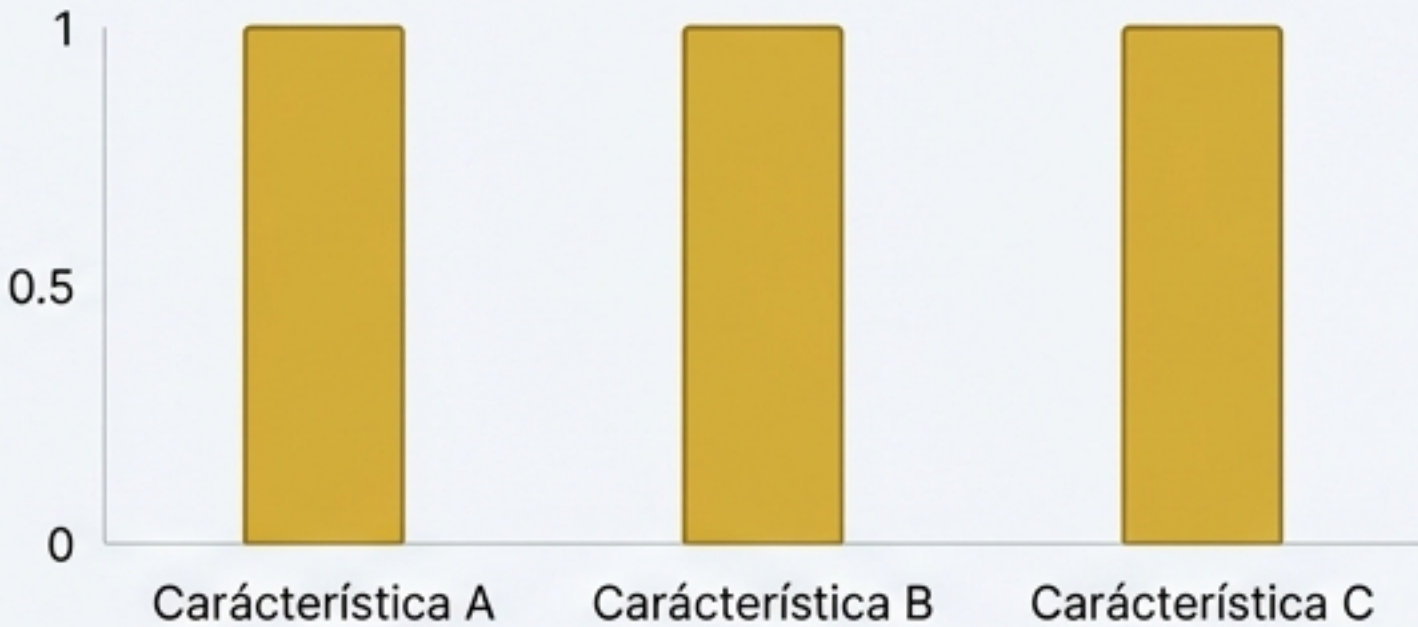
Precisión en prueba:

63%

(Claro sobreajuste y mal rendimiento)



DESPUÉS de Escalar



Precisión en prueba:

97.2%

(Modelo excelente y bien generalizado)



El preprocesamiento no es un paso opcional, es una parte fundamental del flujo de trabajo del modelado.

La Caja de Herramientas del Preprocesamiento

StandardScaler

Transforma los datos para que cada característica tenga media 0 y varianza 1. El más común.

RobustScaler

Usa mediana y cuartiles. Robusto frente a outliers.

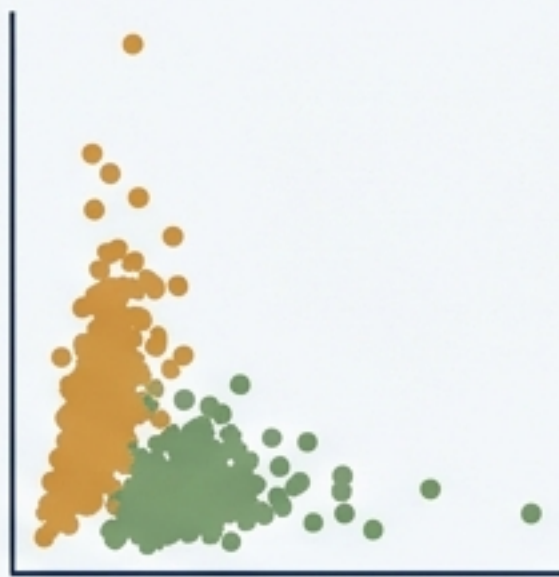
MinmaxScaler

Escala los datos para que todas las características estén entre 0 y 1.

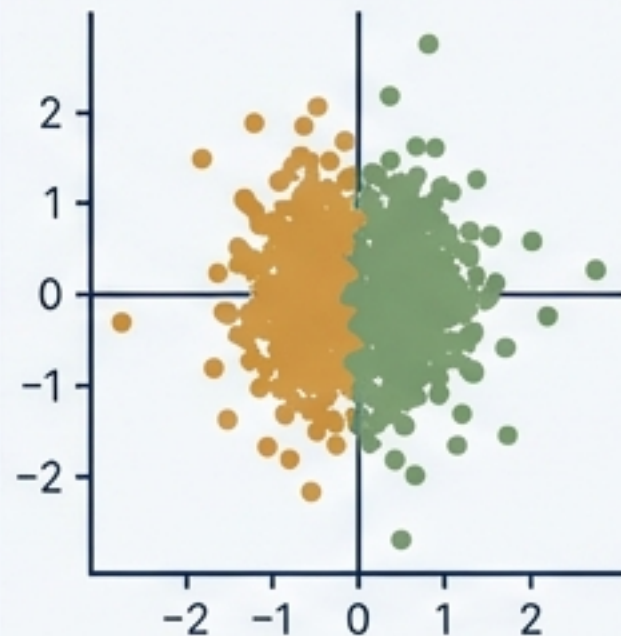
Normalizer

Escala cada *punto de dato* (fila) para que tenga longitud 1.

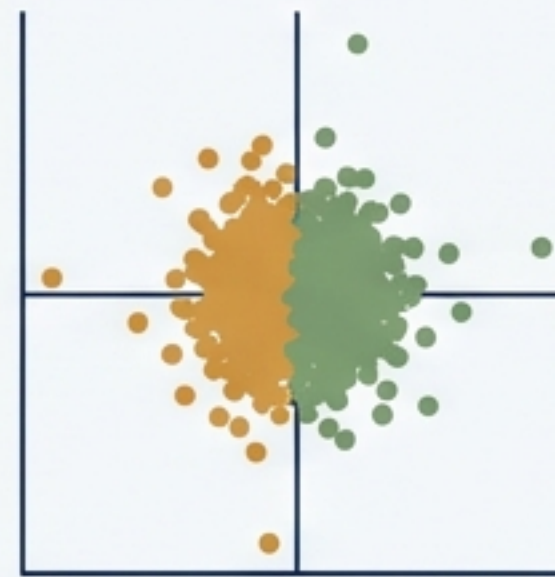
Original Data



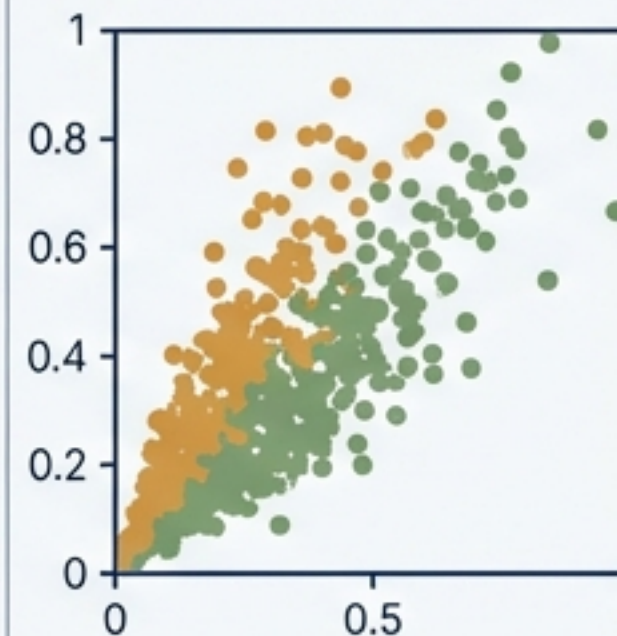
StandardScaler



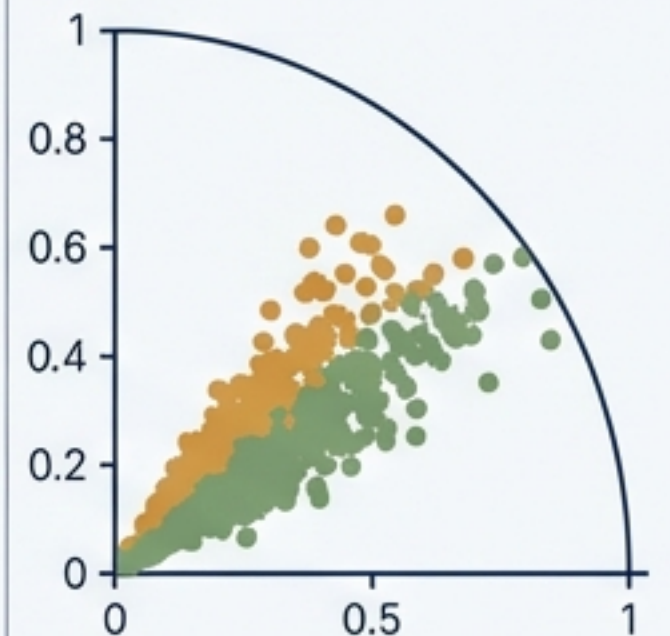
RobustScaler



MinmaxScaler



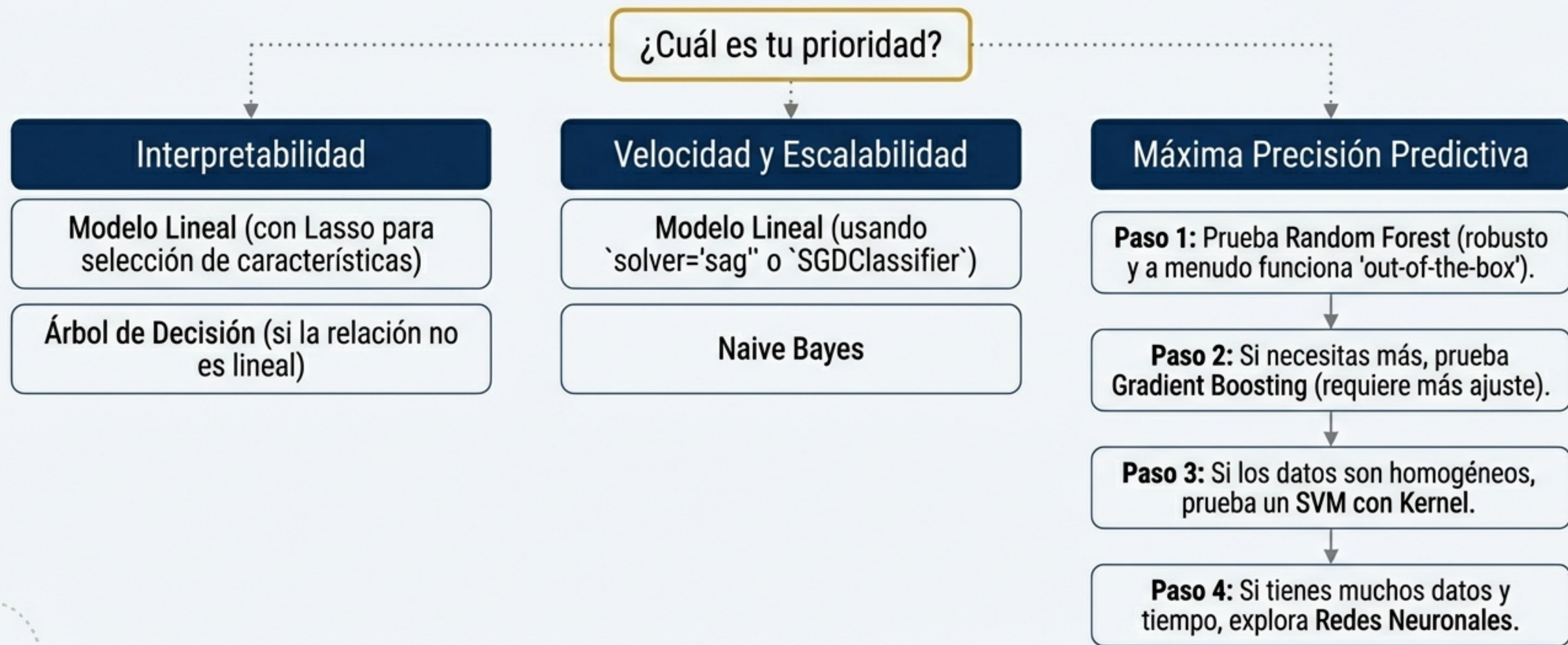
Normalizer



Resumen del Arsenal de Algoritmos

Familia de Algoritmos	Fortalezas Clave	Debilidades	Parámetros a Vigilar	¿Necesita Escalado?
k-Vecinos Cercanos	Fácil de entender, buen baseline.	Lento en predicción, pobre con muchas características.	n_neighbors	Sí
Modelos Lineales	Muy rápidos, alta interpretabilidad, bueno para datos dispersos.	Supuesto de linealidad.	alpha, C	Sí
Árboles de Decisión	Rápido, fácil de visualizar, no requiere escalado.	Tiende a sobreajustar.	max_depth	No
Ensembles	Potentes, robustos, alto rendimiento (Random Forests, Gradient Boosting).	Menos interpretables, pueden consumir más memoria/tiempo.	n_estimators, max_features, learning_rate	No
SVM con Kernel	Potente para límites complejos, funciona bien en datasets medianos.	Sensible a escalado y parámetros, difícil de inspeccionar.	C, gamma	Crítico
Redes Neuronales	Modelos extremadamente complejos, estado del arte en grandes datasets.	Lento de entrenar, requiere mucho dato y tuning.	Capas, nodos, alpha	Crítico

¿Qué Modelo Usar? Una Guía Estratégica



Recuerda: ¡Siempre empieza con un modelo simple como baseline!

El Viaje del Modelado Continúa

Lo que hemos cubierto es el núcleo del aprendizaje supervisado. Esta base es el punto de partida para dominar áreas más avanzadas y construir soluciones de extremo a extremo.


Ingeniería de Pipelines
Características

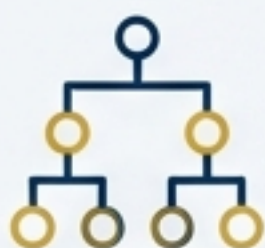

Evaluación
de Modelos


Aprendizaje
No Supervisado


Trabajo con
Datos de Texto



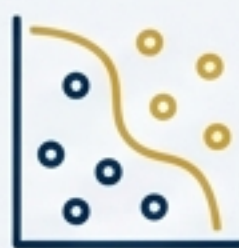
Linear
Models



Decision
Trees



Ensembles



SVMs

Fundamentos del Modelado Supervisado